

第三次编程实践课

软件工程与计算 I · 2026 · 南京大学

| Week 3 · M3 函数与抽象 · 提示词工程进阶

今日安排

知识讲解

Java 函数（方法）

定义 · 调用 · 抽象 · 使用场景

AI实践 M3 提示词工程进阶

精准控制类 · 代码质量类提示词

大作业 Project 02 — Lexer 布置

词法分析器任务讲解与答疑

当堂练习

练习 共 2 题，独立完成

- 练习 1: 读懂函数调用的执行轨迹
- 练习 2: 提取并编写函数

目标：建立对函数调用栈的直觉，学会识别"该提取函数"的时机

知识讲解

Java 函数与抽象 · AI4Coding M3 提示词工程 · 大作业 Project 02

为什么需要函数？

没有函数的世界：

```
// 计算三个矩形的面积之和（复制粘贴三次）
double area1 = 3.0 * 4.0;
double area2 = 5.0 * 6.0;
double area3 = 7.0 * 8.0;
double total = area1 + area2 + area3;
```

有函数的世界：

```
static double area(double w, double h) {
    return w * h;
}
double total = area(3, 4) + area(5, 6) + area(7, 8);
```

函数 = 给一段代码取个名字，让它可以被反复调用。

DRY 原则 (Don't Repeat Yourself)：相同逻辑只写一次。

函数的定义 — 解剖方法签名

```
// 访问修饰符  返回类型      方法名      参数列表
static      double      area  (double width, double height) {
    return width * height;    // 函数体：逻辑实现
}                             // ↑ return 语句：将值返回给调用者
```

组成部分	说明	示例
访问修饰符	谁可以调用	public 、 private 、 static
返回类型	返回值的类型	int 、 double 、 String 、 void
方法名	动词开头，小驼峰	calculateArea 、 printResult
参数列表	输入数据（可为空）	(int n) 、 (String s, int len)
方法体	实现逻辑	{ ... }

void = 没有返回值，方法只做事，不返回数据（如 `System.out.println`）

函数的定义 — 四种常见形式

有参数有返回值（最常见）

```
``java
static int add(int a, int b) {
    return a + b;
}
```

无参数有返回值

```
``java
static double pi() {
    return 3.14159;
}
```

有参数无返回值

```
``java
static void printLine(String msg) {
    System.out.println(">>> " + msg);
}
```

无参数无返回值

```
``java
static void printBanner() {
    System.out.println("=====");
    System.out.println("Welcome!");
    System.out.println("=====");
}
```

选择哪种形式取决于：这段代码需要"输入"吗？需要"输出（返回值）"吗？

函数的调用 — 实参与形参

```
static double circleArea(double radius) { // radius 是形参 (形式参数)
    return 3.14159 * radius * radius;
}
```

// 调用时传入实参 (实际参数)

```
double a = circleArea(5.0); // 5.0 是实参, 传给 radius
double b = circleArea(r);   // 变量 r 的值传给 radius
```

调用的执行过程:

1. 计算实参的值 `circleArea(5.0)`
2. 将实参值复制给形参 `radius = 5.0`
3. 执行函数体 `return 3.14159 * 5.0 * 5.0`
4. 返回值替换整个调用表达式 `→ 78.53975`
5. 继续执行调用处的后续代码

值传递 (Pass by Value): Java 永远传递值的副本, 函数内修改形参不影响实参。

函数的调用 — 值传递的陷阱

基本类型：修改无效 ❌

```
static void doubleIt(int x) {  
    x = x * 2;    // 只改副本!  
}  
  
int n = 5;  
doubleIt(n);  
System.out.println(n);    // 仍然是 5
```

函数内的 `x` 是 `n` 的副本，
修改 `x` 不影响 `n`。

正确做法：用返回值 ✅

```
static int doubled(int x) {  
    return x * 2;  
}  
  
int n = 5;  
n = doubled(n);  
System.out.println(n);    // 10 ✓
```

规律：函数想“改变”某个值，
应该通过返回值把新值传出来，
而不是试图修改形参。

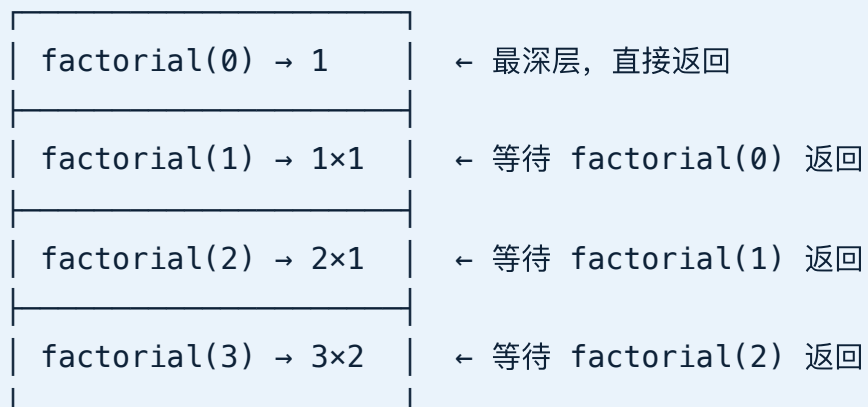
函数的调用 — 调用栈

每次函数调用，JVM 创建一个**栈帧** (Stack Frame)，存放局部变量和返回地址：

```
static int factorial(int n) {  
    if (n == 0) return 1;  
    return n * factorial(n - 1);  
}
```

factorial(3)

调用栈 (从上到下为最新)：



理解调用栈 = 理解程序执行的"记忆机制"，也是调试的核心工具。

变量的作用域与生命周期

作用域 (Scope): 变量可以被访问的代码范围。

生命周期 (Lifetime): 变量在内存中存在的时间段。

局部变量：活在函数内

```
static int add(int a, int b) {  
    int result = a + b; // result 是局部变量  
    return result;  
} // ← result 在这里"消亡"  
  
// System.out.println(result); // ❌ 编译错误!
```

- 局部变量在函数**调用时**创建（随栈帧创建）
- 函数**返回时**销毁（随栈帧弹出）
- 函数外无法访问

同名变量：各自独立

```
static int square(int x) {  
    return x * x; // 这个 x 是 square 的局部变量  
}  
  
static int cube(int x) {  
    return x * x * x; // 这个 x 是 cube 的局部变量  
}
```

两个函数都叫 `x`，互不干扰——
因为它们在**不同的栈帧**中。

核心规律：每次函数调用，都创建独立的"变量空间"；
调用结束，这个空间整体销毁。

函数的抽象 — 何时提取函数? (1/3)

黄金信号：代码出现以下情况时，提取为函数！

信号 1：相同代码出现 ≥ 2 次

// ✗ 复制粘贴

```
System.out.println("Name: " + s1.name);
```

```
System.out.println("Age:  " + s1.age);
```

```
System.out.println("Name: " + s2.name);
```

```
System.out.println("Age:  " + s2.age);
```

// ✓ 提取函数

```
static void printStudent(Student s) {  
    System.out.println("Name: " + s.name);  
    System.out.println("Age:  " + s.age);  
}
```

函数的抽象 — 何时提取函数? (2/3)

黄金信号：代码出现以下情况时，提取为函数！

信号 2：代码块超过 10 行

超过 10 行的代码块通常在做"超过一件事"，
可以拆分为多个具名函数。

函数的抽象 — 何时提取函数? (3/3)

信号 3: 需要写注释解释"做什么"

```
// ✗ 需要注释说明意图
// 判断是否为闰年
if ((y % 4 == 0 && y % 100 != 0)
    || y % 400 == 0) { ... }

// ✔ 函数名就是最好的注释
static boolean isLeapYear(int y) {
    return (y % 4 == 0 && y % 100 != 0)
        || y % 400 == 0;
}
if (isLeapYear(year)) { ... }
```

信号 4: 一段代码可以独立测试

能独立测试 → 职责单一 → 应该是函数

总结: 函数提取的时机 = 重复 / 过长 / 需注释 / 可独立测试——满足一条即可考虑提取。

函数的抽象 — 单一职责原则(1/2)

一个函数只做一件事，只做好一件事。

违反单一职责 ❌

```
static void processAndPrint(int[] arr) {  
    // 做了三件事：求和、求平均、打印  
    int sum = 0;  
    for (int x : arr) sum += x;  
    double avg = (double) sum / arr.length;  
    System.out.println("Sum: " + sum);  
    System.out.println("Avg: " + avg);  
}
```

问题：

- 无法单独复用"求和"逻辑
- 无法在不打印的情况下得到结果
- 测试必须捕获 stdout

函数的抽象 — 单一职责原则(2/2)

一个函数只做一件事，只做好一件事。

遵循单一职责 

```
static int sum(int[] arr) {  
    int total = 0;  
    for (int x : arr) total += x;  
    return total;  
}  
  
static double average(int[] arr) {  
    return (double) sum(arr) / arr.length;  
}  
  
static void printStats(int[] arr) {  
    System.out.println("Sum: " + sum(arr));  
    System.out.println("Avg: " + average(arr));  
}
```

好处:

- 每个函数可独立测试和复用
- 命名清晰, 代码自文档化

函数的抽象 — 命名的艺术

函数名 = 对函数行为的精确描述

命名规则

- 动词开头（描述动作）：`calculate`、`find`、`is`、`has`、`get`、`set`
- 小驼峰命名法：`calculateTax`、`isValidEmail`
- 见名知意：`getStudentById` vs `getData`
- 返回 `boolean` 的函数：用 `is` / `has` / `can` 前缀

好名字 vs 坏名字

坏名字	好名字
<code>f(int x)</code>	<code>factorial(int n)</code>
<code>process(String s)</code>	<code>parseEmail(String s)</code>
<code>check(int n)</code>	<code>isPrime(int n)</code>
<code>do1()</code>	<code>printWelcomeBanner()</code>

参数命名

```
// ❌ 参数名无意义
static double calc(double a, double b) {
    return a * b;
}

// ✅ 参数名传达意图
static double rectangleArea(double width,
                             double height) {
    return width * height;
}
```

一条黄金规则：

如果你需要读函数体才能理解函数做什么，
函数名就取得不够好。

函数的使用场景 — 综合案例(1/2)

任务：读取学生成绩，统计不及格人数、平均分，打印成绩报告。

拆分为函数的设计

```
// 判断是否不及格
static boolean isFailingGrade(int score) {
    return score < 60;
}

// 统计不及格人数
static int countFailing(int[] scores) {
    int count = 0;
    for (int s : scores)
        if (isFailingGrade(s)) count++;
    return count;
}

// 计算平均分
static double average(int[] scores) {
    int sum = 0;
    for (int s : scores) sum += s;
    return (double) sum / scores.length;
}
```

函数的使用场景 — 综合案例(2/2)

```
// 打印单行成绩
static void printScore(int i, int score) {
    String status = isFailingGrade(score)
        ? "✖不及格" : "✔及格";
    System.out.printf("学生%d: %d 分 %s\n",
        i + 1, score, status);
}

// 主函数：只做"编排", 不做具体计算
public static void main(String[] args) {
    int[] scores = {85, 42, 90, 58, 73};
    for (int i = 0; i < scores.length; i++)
        printScore(i, scores[i]);
    System.out.printf("平均分: %.1f\n",
        average(scores));
    System.out.printf("不及格: %d 人\n",
        countFailing(scores));
}
```

main 函数 = 导演, 负责编排;
各功能函数 = 演员, 负责表演。

函数的使用场景 — 与大作业的联系

本次大作业 **Project 02 Lexer** 就是一组精心设计的函数：

你将实现的函数

```
// 主任务：字符串 → Token 列表
ArrayList<Token> lex(String input)

// 扩展 1：统计各类型 Token 数量
String countTokens(ArrayList<Token> tokens)

// 扩展 2：Token 列表 → 字符串
String tokensToString(ArrayList<Token> tokens)
```

函数设计的体现

设计原则	在 Lexer 中的体现
单一职责	<code>lex</code> 只做分词，不做统计
见名知意	<code>countTokens</code> 名称即说明
参数明确	输入/输出类型精确定义
可独立测试	每个方法有独立测试用例

这三个函数相互独立、职责清晰——这正是好的函数设计。

AI4Coding M3

| 提示词工程进阶：精准控制 · 代码质量

AI4Coding 实践 — M3 提示词工程进阶

核心问题 如何让 AI 生成符合规范、质量可靠的代码，而不只是"能跑"的代码？

上节回顾：基础四要素

要素	说明
角色	你是一位 Java 工程师
任务	写一个方法
约束	只用 for 循环
示例	输入/输出样例

基础四要素让 AI 理解你要什么。

本节进阶：两类提示词

精准控制类

- 精确指定输出的技术细节
- 控制 AI "怎么做"

代码质量类

- 要求注释、异常处理、可读性
- 控制 AI 代码的"好不好"

AI 生成的代码默认"能跑"，
但"能跑"≠"好代码"。
这两类提示词负责弥补这个差距。

10 个实用提示词 — 精准控制类

核心思想：精准控制类 Prompt 告诉 AI 怎么改——防止 AI 改动范围失控。

#	提示词模板	适用场景
1	"仅修复此问题，不要修改其他代码"	防止 AI 改动范围过大
2	"请使用最小修改来解决此问题"	保持现有代码风格一致
3	"换个思路来解决此问题"	当前方案性能或逻辑不满意
4	"仔细阅读全文，检查是否还有其他同类问题"	让 AI 做全局自我审查
5	"请为我梳理当前代码的整体逻辑"	理解他人代码或遗留代码

💡 最常用：No.1 和 No.2——能避免 AI 生成"牵一发动全身"的修改。

精准控制类 — 场景对比

No.1 — 只修复这一个问题

场景：发现 `lex()` 方法处理负整数时有 Bug

✗ 不好的 Prompt:

"帮我修复 `Lexer.java`"

→ AI 可能重写整个方法，破坏已通过的测试

✓ 精准 Prompt:

"在以下代码中，仅修复负整数解析的问题，

不要修改其他任何逻辑：

[粘贴 `lex()` 方法]"

No.4 — 全局自我审查

场景：担心同类问题还有遗漏

✓ Prompt:

"仔细阅读以下 `Lexer` 实现，

检查是否还有其他类似 '负整数判断'

的边界处理问题，列出所有你发现的问题：

[粘贴代码]"

No.5 — 梳理逻辑

✓ 理解别人的代码:

"请为我梳理 `CharLookaheadStream`

的整体工作原理，

用中文一步一步解释"

10 个实用提示词 — 代码质量类

核心思想：代码质量类 Prompt 告诉 AI 代码要**好在哪里**——"能跑"只是起点。

#	提示词模板	适用场景
6	"请提供详细的解释和行内代码注释"	生成可维护的文档
7	"生成测试用例来验证此更改，包含正常路径和异常路径"	自动化测试覆盖
8	"增加详细的日志输出，方便追溯错误"	调试辅助
9	"请简化代码逻辑，去除冗余部分"	代码瘦身
10	"优化代码结构，使其更加模块化，方便单元测试"	重构升级

💡 No.7 组合 No.8 特别有效：让 AI 生成带日志的测试用例，调试时事半功倍。

代码质量类 — 场景对比(1/2)

No.7 + No.8 组合：带日志的测试

"为以下 countTokens 方法：

1. 生成 JUnit 4 测试，覆盖正常路径和异常路径
2. 在测试方法中增加日志输出，
打印每个 Token 的类型，方便定位失败原因

[粘贴 countTokens 代码]"

AI 生成的测试示例：

```
@Test
public void testCountBasic() {
    var tokens = lexer.lex("(+ 1 2)");
    System.out.println("Tokens: " + tokens); // No.8
    String result = lexer.countTokens(tokens);
    assertEquals("整数:2 符号:1 ...", result);
}
```

代码质量类 — 场景对比(2/2)

No.9 — 简化冗余代码

"请简化以下代码逻辑，去除冗余部分，
但不改变其功能：

```
if (t instanceof IntegerToken) {  
    intCount = intCount + 1;  
} else if (t instanceof SymbolToken) {  
    symCount = symCount + 1;  
} else if ..."
```

No.10 — 模块化重构

"优化 lex() 方法结构，
使其更加模块化：
将读取整数、字符串、符号
各提取为独立的私有方法，
方便单元测试每个子功能"

AI4Coding 实践任务

任务 用提示词控制 AI，为大作业 Lexer 的三个方法生成高质量代码：

Step 1: 精准控制生成 `lex()`

使用以下要素构造 Prompt：

- **方法签名：** `ArrayList<Token> lex(String input)`
- **算法约束：** 用 `CharLookaheadStream`，贪心匹配
- **边界条件：** 空字符串、只有空格、嵌套括号
- **禁止：** 不能用正则表达式 `split`

你的 Prompt：

Step 2: 代码质量 — 让 AI 审查你的代码

把你写的（或 AI 生成的）`countTokens` 粘贴给 AI：

"请审查以下 Java 方法的代码质量：
1. 变量命名是否清晰？
2. 是否有可以简化的逻辑？
3. 格式字符串是否可能出错？
4. 给出改进建议并提供改进版本"

Step 3: 填表反思

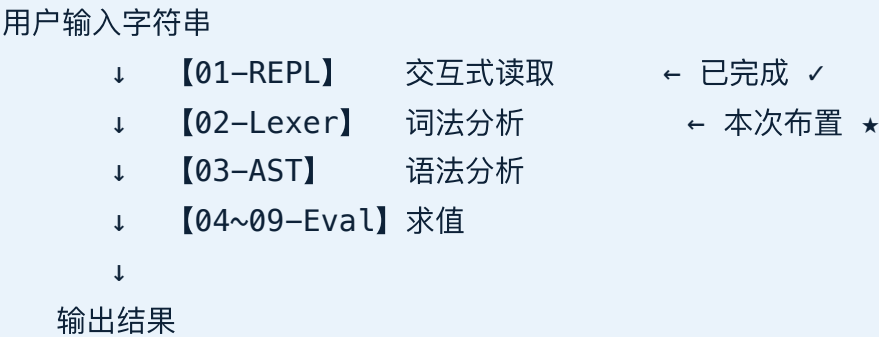
维度	精准控制效果	代码质量效果
生成准确性		
需要几轮追问		
最难控制的点		

大作业 Project 02

| **Lexer — 词法分析器**

大作业进度回顾 (1/5)

解释器流水线



9 个子项目概览

#	项目	核心内容
01	REPL	循环读取 + 回显
02	Lexer	字符串 → Token
03	AST	Token → 语法树
04	Eval	基础求值
05	Env	词法作用域
06	Control	闭包 + 条件
07	TCO	尾调用优化
08	Atom	动态求值 + 可变状态
09	Quote	元编程

大作业进度回顾 (2/5)

Project 01 完成情况

- **基础任务**：REPL 循环读取并回显 ✓
- **扩展1**：行号提示 `[n] >` ✓
- **扩展2**：退出时统计处理行数 ✓
- **截止时间**：已过，未提交请尽快补交

参考答案设计解析：

- 采用 **READ-EVAL-PRINT-RE 四阶段分离** 架构
- `READ(str)`：直接返回输入（暂不解析）
- `EVAL(ast, env)`：直接返回AST（暂不求值）
- `PRINT(exp)`：由主循环负责实际输出
- `RE(env, str)`：组合READ+EVAL形成处理链
- **三个版本演进**：基础版 → 行号提示 → 会话统计

大作业进度回顾 (3/5)

```
public static void REPL() {
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    while (true) {
        String line;
        try {
            line = br.readLine();
            if (line == null) { continue; }
            else if (line.equals("exit")) { break; }
        } catch (IOException e) {
            System.out.println("IOException: " + e.getMessage());
            break;
        }
        System.out.println(PRINT(RE(null, line)));
    }
}
```

大作业进度回顾 (4/5)

```
// 扩展 1: 每次输入前打印行号提示 [n] >
public static void REPL1() {
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    int lineNum = 1;

    while (true) {
        System.out.print "[" + lineNum + "] > ";
        lineNum++;
        String line;
        try {
            line = br.readLine();
            if (line == null) { continue; }
            else if (line.equals("exit")) { break; }
        } catch (IOException e) {
            System.out.println("IOException: " + e.getMessage());
            break;
        }

        System.out.println(PRINT(RE(null, line)));
    }
}
```

大作业进度回顾 (5/5)

// 扩展 2: 退出时输出本次会话共处理的行数

```
public static void REPL2() {
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    int count = 0;

    while (true) {
        String line;
        try {
            line = br.readLine();
            if (line == null) { continue; }
            else if (line.equals("exit")) { break; }
        } catch (IOException e) {
            System.out.println("IOException: " + e.getMessage());
            break;
        }

        count++;
        System.out.println(PRINT(RE(null, line)));
    }

    System.out.println("共处理 " + count + " 行输入, 再见! ");
}
```

Project 02 — Lexer 背景与目标

词法分析 (Lexical Analysis): 将字符串切分为有意义的词法单元 (Token)。

输入字符串: "(+ 1 -2)"

词法分析后: LeftParen SymbolToken(+) IntegerToken(1) IntegerToken(-2) RightParen
 (+ 1 -2)

为什么需要词法分析?

- 计算机不懂 "(+ 1 2)" 这个字符串的含义
- 词法分析是"理解"的第一步: 识别出**有意义的单词**
- 之后的语法分析 (AST) 才能识别**结构**, 求值器才能**计算**

类比: 阅读一句话前, 先要认出每个**词**; 程序分析前, 先要认出每个 **Token**。

Project 02 — Token 类型规范

五种 Token 类型

类型	示例	识别规则
整数 IntegerToken	5 、 -42	可选负号 + 数字序列
字符串 StringToken	"hello"	双引号包裹，支持 <code>\\</code> <code>\</code> 转义
左括号 LeftParen	(	字符 (
右括号 RightParen	)	字符)
符号 SymbolToken	+ 、 foo 、 let*	不是数字、不含 "()" 空格的字符序列

Token 类层次（已提供）

- Token ← 抽象基类
 - IntegerToken ← 已实现
 - StringToken ← 已实现
 - SymbolToken ← 已实现
 - LeftParen ← 已实现
 - RightParen ← 已实现

你只需要实现 `Lexer.java` ，
Token 类本身已全部提供，直接使用即可。

贪心匹配原则：每个 Token 尽可能长——
`-42` 是一个 IntegerToken，不是 `-` + `42` 。

Project 02 — CharLookaheadStream

`CharLookaheadStream` 是字符流工具，提供**前瞻（Lookahead）**能力：

```
CharLookaheadStream input = new CharLookaheadStream("(+ 1 2)");
```

方法	作用	示例
<code>input.eof()</code>	是否到达字符串末尾	<code>false</code> （还有字符）
<code>input.lookahead(1)</code>	预览下一个字符，不移动指针	<code>'('</code>
<code>input.consume()</code>	读取下一个字符，指针右移	<code>'('</code> ，指针移到 <code>+</code>

为什么需要前瞻？

读到 `'-'` 时，还不知道它是符号 `'-'` 还是负整数的开头

- 用 `lookahead(1)` 看下一个字符是否是数字
- 是数字：读取整数 Token
- 不是数字：当作符号 Token

前瞻让你"看一步"后再决定——这是词法分析器的核心技巧。

Project 02 — 任务一：lex() 主循环(1/2)

```
ArrayList<Token> lex(String input) {  
    CharLookaheadStream stream =  
        new CharLookaheadStream(input);  
    ArrayList<Token> tokens = new ArrayList<>();  
  
    while (!stream.eof()) {  
        char c = (char) stream.lookahead(1);  
        if (Character.isWhitespace(c)) {  
            stream.consume();           // 跳过空白  
        } else if (c == '(') {  
            stream.consume();  
            tokens.add(new LeftParen());  
        } else if (c == ')') {  
        } else if (c == '"') {  
        } else if (isIntegerStart(c, stream)) {  
        } else {  
        }  
    }  
    return tokens;  
}
```

关键： `lookahead(1)` 只预览，不消耗； `consume()` 才移动指针。

Project 02 — 任务一：辅助函数与示例(2/2)

辅助判断：整数的开头

```
// c 是数字, 或者 c=='-' 且后面跟着数字
boolean isIntegerStart(char c,
                        CharLookaheadStream s) {
    if (Character.isDigit(c)) return true;
    if (c == '-') {
        int next = s.lookahead(2); // 需要看两步!
        return next != -1
            && Character.isDigit((char) next);
    }
    return false;
}
```

逐字符解析示例

```
输入: (+ 1 -2)
c='(' → LeftParen
c=' ' → 跳过
c='+' → SymbolToken(+)
c=' ' → 跳过
c='1' → IntegerToken(1)
c=' ' → 跳过
```

Project 02 — 任务二：countTokens()

统计 Token 列表中各类型的数量，返回格式**严格**为：

整数:N 符号:N 字符串:N 左括号:N 右括号:N

```
String countTokens(ArrayList<Token> tokens) {  
    int intCount = 0, symCount = 0, strCount = 0;  
    int lpCount = 0, rpCount = 0;  
    for (Token t : tokens) {  
        if (t instanceof IntegerToken) intCount++;  
        else if (t instanceof SymbolToken) symCount++;  
        // ... 补充其余分支  
    }  
    return String.format(  
        "整数:%d 符号:%d 字符串:%d 左括号:%d 右括号:%d",  
        intCount, symCount, strCount, lpCount, rpCount);  
}
```

注意：冒号是中文全角 **：**，格式严格匹配，否则测试不通过。

Project 02 — 任务三: tokensToString()

将 Token 列表转回字符串, Token 之间用单个空格分隔:

```
String tokensToString(ArrayList<Token> tokens) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < tokens.size(); i++) {
        if (i > 0) sb.append(" ");
        sb.append(tokenToStr(tokens.get(i)));
    }
    return sb.toString();
}
```

各类型转换规则

Token	输出
IntegerToken(n)	"-42"
StringToken(s)	"\"hello\""
SymbolToken(x)	"+"
LeftParen	"("
RightParen	")"

Project 02 — 测试用例说明

运行测试

```
mvn test
```

基础测试 (lex 方法)

测试	输入	期望
testAdd	"(+ 1 2)"	[(+ 1 2)]
testNeg	"(- 3 -2)"	[(- 3 -2)]
testString	"\"hi\""	[StringToken(hi)]
testNested	"(+ (* 2 3) 4)"	多层嵌套

扩展测试

测试	验证内容
test_ext1_count_basic	(+ 1 2) 的计数
test_ext1_count_empty	空列表计数 (全为0)
test_ext1_count_with_string	含 StringToken 的计数
test_ext2_toString_simple	(+ 1 2) 转字符串
test_ext2_toString_with_neg	含负整数转字符串
test_ext2_toString_empty	空列表转空字符串

全部绿色 = 作业完成 ✓

通过率将记入平时分；
代码风格（命名、注释）也会评分。

课后作业

必做

1. 完成 Project 02 Lexer

提交到 SEECODER-Coder，截止下次课前。

三个方法都需完成：`lex`、`countTokens`、`tokensToString`

2. AI 辅助实验报告（一段话）

记录你用了哪类提示词（精准控制/代码质量），
AI 在哪里帮到了你，在哪里出错了

3. 函数抽象练习

把之前 Project 01 中较长的代码块，
尝试提取为至少 2 个命名函数，
提交重构后的版本

选做

- 思考：`lex` 方法中有没有可以进一步提取的辅助函数？
试着重构，让每个函数不超过 15 行
- 阅读 `CharLookaheadStream.java` 的源码，理解前瞻流的实现原理
- 尝试用 AI 生成 `lex` 的一个错误版本，再自己找出 Bug

预习

下周内容：类与对象（OOP 入门）、继承基础

带着问题来：`Token` 的类层次是怎么设计的？

`Token`、`IntegerToken`、`SymbolToken`
之间是什么关系？

当堂练习

当堂练习 · 独立完成 · 25 分钟

练习 1 — 读懂函数调用轨迹（阅读）

15 min 阅读以下代码，**手动追踪每一步调用**，再翻到下一页回答问题：

```
public class Trace {

    static int double_val(int x) {
        return x * 2;
    }

    static int sumDoubled(int a, int b) {
        int da = double_val(a);
        int db = double_val(b);
        return da + db;
    }

    static String classify(int n) {
        if (n > 100) return "大";
        if (n > 10)  return "中";
        return "小";
    }

    public static void main(String[] args) {
        int result = sumDoubled(7, 13);
        System.out.println(result);
        System.out.println(classify(result));
    }
}
```

练习 1 — 读懂函数调用轨迹（答题）

① 填写调用轨迹表：

调用	传入参数	返回值
double_val(7)	x = ?	?
double_val(13)	x = ?	?
sumDoubled(7, 13)	a=7, b=13	?
classify(result)	n = ?	?

② main 最终打印了什么？（两行）

③ 如果把 `sumDoubled` 改为：

```
static int sumDoubled(int a, int b) {  
    a = a * 2;  
    b = b * 2;  
    return a + b;  
}
```

结果有变化吗？为什么？

④ 如果把 `classify` 的第一个 `if` 改为 `if (n >= 40)`，`classify(result)` 返回什么？

⑤ 画出 `sumDoubled(7, 13)` 执行时的调用栈（三层）

练习 2 — 提取并编写函数

10 min

题目 A（必做）

下面的代码重复了三次相同的逻辑：

```
// 打印学生信息（重复了三次）
System.out.println("姓名: Alice");
System.out.println("分数: 85");
System.out.println("等级: " + (85 >= 60 ? "及格" : "不及格"));
System.out.println("----");

System.out.println("姓名: Bob");
System.out.println("分数: 45");
System.out.println("等级: " + (45 >= 60 ? "及格" : "不及格"));
System.out.println("----");

System.out.println("姓名: Carol");
System.out.println("分数: 92");
System.out.println("等级: " + (92 >= 60 ? "及格" : "不及格"));
System.out.println("----");
```

要求：提取一个函数消除重复，并用函数调用替换以上代码。

题目 B（选做）

实现以下两个函数：

```
// 判断一个整数是否为完全数
// 完全数：所有真因子之和等于自身
// 如：6 的真因子是 1,2,3, 和为 6 → 完全数
// 如：28 = 1+2+4+7+14 → 完全数
static boolean isPerfect(int n)

// 找出 1 到 max 之间所有完全数
// 返回它们的个数
static int countPerfects(int max)
```

示例：

```
isPerfect(6) → true
isPerfect(7) → false
countPerfects(30) → 2    (6 和 28)
```

提示：`isPerfect` 里用循环求真因子之和，
`countPerfects` 里调用 `isPerfect`。

附录

JUnit 测试 · 大作业提交规范

附录 — 如何运行 Project 02 的测试

环境确认

```
# 确认 Java 版本 (需要 11+)
java -version

# 确认 Maven 已安装
mvn -version
```

运行全部测试

```
cd 02-Lexer
mvn test
```

只运行某一个测试

```
mvn test -Dtest=LexerTest#testAdd
```

测试输出解读

```
[INFO] Tests run: 13, Failures: 0, Errors: 0
```

字段	含义
Tests run	运行的测试总数
Failures	断言失败数 (输出不符合预期)
Errors	异常数 (代码抛出了意外异常)

目标: Failures = 0, Errors = 0

常见错误排查

- NullPointerException : 方法返回了 null
- AssertionError : 输出格式不对 (如冒号全/半角)
- IndexOutOfBoundsException : 访问了越界位置

附录 一 大作业提交规范

提交内容

1. 代码: `Lexer.java` (只提交这一个文件)
2. 提交平台: SEECODER-Coder 系统
3. 截止时间: 下次课开始前

代码规范要求

- 函数命名遵循小驼峰
- 关键逻辑有行内注释 (中文即可)
- 无调试用的 `System.out.println`
- 不修改 `Token` 类及其子类

AI 使用说明

- ✓ 允许使用 AI 辅助理解题意、调试
- ✓ 允许用 AI 生成代码片段后自己修改
- ✗ 不允许直接粘贴 AI 生成的完整解答
- ✗ 不允许与他人共享代码

评分时会结合代码理解的课堂抽查。

第三次编程实践课

软件工程与计算 I · 2026 · 南京大学

| 感谢聆听 · 有问题随时提问